LAPORAN

WORKSHOP SISTEM INFORMASI WEB SERVER (TIF330805)

SEMESTER 3



RESTFUL API DAN ENDPOINT REQUEST/RESPONSE

Disusun oleh :

Mochammad Syifaul Qulub (E41250219)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Application Programming Interface (API) merupakan mekanisme yang memungkinkan dua aplikasi atau sistem berkomunikasi dan bertukar data. Dalam pengembangan aplikasi web, API berperan sebagai penghubung antara client dan server untuk melakukan operasi pengolahan data melalui permintaan (*request*) dan respons (*response*).

RESTful API merupakan API yang menerapkan prinsip Representational State Transfer (REST). RESTful API umumnya menggunakan protokol HTTP dengan metode GET, POST, PUT, dan DELETE untuk mengambil, menambahkan, memperbarui, serta menghapus data. Data yang dikirimkan maupun diterima dapat menggunakan format JSON (*JavaScript Object Notation*).

Pada praktikum Acara 16, dilakukan pengembangan RESTful API menggunakan PHP dan MySQL untuk mengelola data mahasiswa. Praktikum ini mencakup pembuatan endpoint, pemrosesan request dan response, penerapan validasi input, penggunaan HTTP status code, serta pengujian kondisi berhasil maupun gagal.

1.2 Tujuan Praktikum

Tujuan pelaksanaan praktikum ini adalah:

1. Memahami konsep RESTful API dan mekanisme komunikasi client-server.
2. Mengimplementasikan endpoint API menggunakan PHP dan MySQL.
3. Menerapkan metode HTTP GET, POST, PUT, dan DELETE.
4. Mengolah request dan response menggunakan format JSON.
5. Menerapkan validasi input dan penanganan kesalahan.
6. Menggunakan HTTP status code sesuai hasil pemrosesan request.
7. Melakukan pengujian endpoint serta menyiapkan dokumentasi API.

BAB 2. DASAR TEORI

2.1 RESTful API

RESTful API adalah antarmuka pemrograman aplikasi yang menerapkan prinsip REST untuk menyediakan akses terhadap sumber daya (*resource*) melalui protokol HTTP. Sumber daya dalam API dapat berupa data mahasiswa, dosen, produk, dan berbagai jenis data lainnya.

Dalam praktikum ini, data mahasiswa menjadi resource utama yang diakses melalui endpoint /api/mahasiswa. API menerima request dari client, memprosesnya menggunakan PHP, berinteraksi dengan database MySQL, kemudian mengembalikan response dalam format JSON.

2.2 HTTP Method

HTTP method digunakan untuk menentukan operasi yang dilakukan terhadap suatu resource. Metode yang digunakan dalam praktikum ini meliputi:

1. **GET**, digunakan untuk mengambil seluruh data mahasiswa atau data mahasiswa berdasarkan ID.
2. **POST**, digunakan untuk menambahkan data mahasiswa baru.
3. **PUT**, digunakan untuk memperbarui seluruh data mahasiswa berdasarkan ID.
4. **DELETE**, digunakan untuk menghapus data mahasiswa berdasarkan ID.

2.3 Endpoint, Request, dan Response

Endpoint merupakan alamat yang digunakan client untuk mengakses resource tertentu pada API. Request adalah permintaan yang dikirimkan client kepada server, sedangkan response adalah jawaban yang dikirimkan server setelah memproses permintaan tersebut.

Request dapat memuat header dan body. Pada operasi POST dan PUT, body digunakan untuk mengirim data mahasiswa dalam format JSON. Response API pada praktikum ini memiliki struktur yang terdiri atas:

- success, menunjukkan keberhasilan atau kegagalan operasi.

- message, berisi keterangan hasil pemrosesan.
- data, berisi data hasil operasi atau nilai null apabila tidak tersedia.

2.4 HTTP Status Code

HTTP status code menunjukkan hasil pemrosesan request oleh server. Beberapa status code yang relevan dengan praktikum ini adalah:

1. **200 OK**, menandakan request berhasil diproses.
2. **201 Created**, menandakan resource baru berhasil dibuat.
3. **400 Bad Request**, menandakan request tidak valid.
4. **404 Not Found**, menandakan resource yang diminta tidak ditemukan.
5. **405 Method Not Allowed**, menandakan metode HTTP tidak didukung.
6. **409 Conflict**, menandakan terjadi konflik data, seperti NIM yang sudah terdaftar.
7. **500 Internal Server Error**, menandakan terjadi kesalahan pada server.

2.5 JSON (JavaScript Object Notation)

JSON merupakan format pertukaran data berbasis teks yang banyak digunakan dalam komunikasi API. JSON memiliki struktur sederhana berupa pasangan key dan value sehingga mudah diproses oleh berbagai bahasa pemrograman.

Dalam praktikum ini, JSON digunakan untuk mengirim data mahasiswa melalui request POST dan PUT serta mengembalikan hasil pemrosesan API kepada client.

BAB 3. IMPLEMENTASI DAN PENGUJIAN

3.1 Persiapan Lingkungan

Praktikum ini menggunakan beberapa perangkat lunak dan teknologi sebagai berikut:

1. Sistem operasi Windows.
2. Laragon sebagai lingkungan server lokal.
3. PHP untuk mengimplementasikan RESTful API.
4. MySQL sebagai sistem manajemen basis data.
5. phpMyAdmin untuk mengelola dan mengekspor database.
6. Visual Studio Code sebagai editor kode.
7. PowerShell untuk melakukan pengujian request API.
8. Postman Collection dalam format JSON untuk dokumentasi request API.

3.2 Struktur Folder Proyek

Proyek dibuat pada folder ACARA 16 Restful API endpoint request response. Struktur folder yang digunakan adalah sebagai berikut:

ACARA 16 Restful API endpoint request response/
├── .htaccess
├── api/
│ └── mahasiswa.php
├── config/
│ └── database.php
├── postman/
│ └── Acara16.postman_collection.json
└── acara_16.sql

Keterangan:

1. .htaccess digunakan untuk mengatur URL endpoint dan meneruskan request ke file API.
2. api/mahasiswa.php berisi logika pemrosesan request GET, POST, PUT, dan DELETE serta validasi data.

3. config/database.php berisi konfigurasi koneksi database MySQL.
4. postman/Acara16.postman_collection.json berisi kumpulan request API untuk dokumentasi dan pengujian melalui Postman.
5. acara_16.sql merupakan file hasil ekspor database yang digunakan untuk menyimpan struktur tabel dan data mahasiswa.

3.3 Implementasi Database

Database yang digunakan dalam praktikum ini bernama acara 16. Di dalam database tersebut dibuat tabel mahasiswa untuk menyimpan data mahasiswa.
Tabel mahasiswa memiliki empat kolom, yaitu:

1. id, sebagai primary key dengan nilai yang bertambah otomatis.
2. nim, untuk menyimpan nomor induk mahasiswa dan memiliki batasan unik.
3. nama, untuk menyimpan nama mahasiswa.
4. email, untuk menyimpan alamat email mahasiswa.

Data awal yang digunakan dalam pengujian terdiri atas tiga mahasiswa, yaitu Budi, Siti, dan Andi. Database kemudian digunakan untuk menguji proses penambahan, pembaruan, pengambilan, dan penghapusan data.

3.4 Implementasi RESTful API

RESTful API diimplementasikan menggunakan PHP dengan koneksi database MySQL melalui PDO (*PHP Data Objects*). PDO digunakan untuk menjalankan query database menggunakan prepared statement sehingga nilai input dapat dikirimkan sebagai parameter query.
API menyediakan endpoint berikut:

No.	HTTP Method	Endpoint	Fungsi
1	GET	/api/mahasiswa	Mengambil seluruh data mahasiswa
2	GET	/api/mahasiswa/{id}	Mengambil mahasiswa berdasarkan ID
3	POST	/api/mahasiswa	Menambahkan mahasiswa baru
4	PUT	/api/mahasiswa/{id}	Memperbarui data mahasiswa
5	DELETE	/api/mahasiswa/{id}	Menghapus data mahasiswa

Pada metode POST dan PUT, API melakukan validasi terhadap kelengkapan NIM, nama, dan email serta memeriksa format email. API juga memeriksa kemungkinan duplikasi NIM sebelum melakukan penyimpanan atau pembaruan data.

Jika request berhasil, API mengembalikan response JSON dengan status code yang sesuai. Apabila data tidak ditemukan atau input tidak valid, API mengembalikan pesan kesalahan yang menjelaskan penyebab kegagalan.

3.5 Pengujian Endpoint

Pengujian dilakukan menggunakan PowerShell dengan mengirimkan request HTTP ke endpoint API melalui server lokal Laragon.

3.5.1 Pengujian GET Seluruh Data Mahasiswa

Pengujian GET dilakukan untuk mengambil seluruh data mahasiswa yang tersimpan dalam database. Hasil pengujian menunjukkan bahwa API mengembalikan daftar mahasiswa beserta atribut ID, NIM, nama, dan email dalam format JSON.


```
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": [
    {
      "id": 1,
      "nim": "23001",
      "nama": "Budi Santoso",
      "email": "budi.santoso@gmail.com"
    },
    {
      "id": 2,
      "nim": "23002",
      "nama": "Siti",
      "email": "siti@gmail.com"
    },
    {
      "id": 3,
      "nim": "23003",
      "nama": "Andi",
      "email": "andi@gmail.com"
    }
  ]
}
```

3.5.2 Pengujian GET Berdasarkan ID

Pengujian ini dilakukan dengan mengakses endpoint /api/mahasiswa/1. API berhasil mengembalikan data mahasiswa dengan ID 1, yaitu Budi, beserta NIM dan alamat emailnya.


```
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": {
    "id": 1,
    "nim": "23001",
    "nama": "Budi Santoso",
    "email": "budi.santoso@gmail.com"
  }
}
```


3.5.3 Pengujian POST

Pengujian POST dilakukan dengan mengirimkan data mahasiswa baru melalui body request dalam format JSON. Data yang digunakan adalah NIM 23004, nama Dewi, dan email dewi@gmail.com.

Hasil pengujian menunjukkan bahwa data berhasil ditambahkan ke database. API mengembalikan pesan keberhasilan dan status HTTP 201 Created beserta data mahasiswa yang baru dibuat.


```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> $body = @{
>>     nim = "23004"
>>     nama = "Dewi"
>>     email = "dewi@gmail.com"
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod -Uri $uri -Method POST -ContentType "applica
tion/json" -Body $body | ConvertTo-Json -Depth 5
{
    "success":  true,
    "message":  "Data mahasiswa berhasil ditambahkan",
    "data":  {
                 "id":  4,
                 "nim":  "23004",
                 "nama":  "Dewi",
                 "email":  "dewi@gmail.com"
             }
}
```

3.5.4 Pengujian PUT

Pengujian PUT dilakukan untuk memperbarui data mahasiswa dengan ID 1. Nama mahasiswa diubah menjadi Budi Santoso dan alamat email diperbarui menjadi budi.santoso@gmail.com.

Hasil pengujian menunjukkan bahwa data berhasil diperbarui. API mengembalikan pesan keberhasilan dan data mahasiswa setelah perubahan.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> $body = @{
>>     nim = "23001"
>>     nama = "Budi Santoso"
>>     email = "budi.santoso@gmail.com"
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod -Uri "$uri/1" -Method PUT -ContentType "appl
ication/json" -Body $body | ConvertTo-Json -Depth 5
{
    "success":  true,
    "message":  "Data mahasiswa berhasil diperbarui",
    "data":  {
                 "id":  1,
                 "nim":  "23001",
                 "nama":  "Budi Santoso",
                 "email":  "budi.santoso@gmail.com"
             }
}
```

3.5.5 Pengujian DELETE

Pengujian DELETE dilakukan dengan menghapus data mahasiswa yang memiliki ID 4, yaitu data Dewi yang sebelumnya ditambahkan melalui POST.

Hasil pengujian menunjukkan bahwa data berhasil dihapus dari database dan API mengembalikan pesan keberhasilan beserta ID data yang dihapus.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> Invoke-RestMethod -Uri
 "$uri/4" -Method DELETE | ConvertTo-Json -Depth 5
{
    "success":  true,
    "message":  "Data mahasiswa berhasil dihapus",
    "data":  {
                 "id":  4
             }
}
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response>
```

3.6 Pengujian Validasi dan Error

Selain menguji operasi utama, dilakukan pengujian terhadap request yang tidak valid untuk memastikan API dapat menangani kesalahan dengan baik.

3.6.1 Pengujian ID Tidak Ditemukan

Pengujian dilakukan dengan meminta data mahasiswa menggunakan ID 999. Karena data tersebut tidak ditemukan, API mengembalikan status HTTP 404 Not Found dengan pesan Data mahasiswa tidak ditemukan.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> Invoke-RestMethod -Uri
 "$uri/999" -Method GET
Invoke-RestMethod :
{
    "success": false,
    "message": "Data mahasiswa tidak ditemukan",
    "data": null
}
At line:1 char:1
+ Invoke-RestMethod -Uri "$uri/999" -Method GET
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.Htt
   pWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcepti
  on
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Mic
   rosoft.PowerShell.Commands.InvokeRestMethodCommand
```

3.6.2 Pengujian NIM Kosong

Pengujian dilakukan dengan mengirimkan data mahasiswa yang memiliki NIM kosong. API menolak request tersebut karena NIM, nama, dan email wajib diisi. Respons yang dikembalikan adalah status HTTP 400 Bad Request.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> $body = @{
>>     nim = ""
>>     nama = "Test"
>>     email = "test@gmail.com"
>> } | ConvertTo-Json
>>
>> try {
>>     Invoke-RestMethod -Uri $uri -Method POST -ContentType "app
lication/json" -Body $body
>> } catch {
>>     $_.ErrorDetails.Message
>> }

{
    "success": false,
    "message": "NIM, nama, dan email wajib diisi",
    "data": null
}
```

3.6.3 Pengujian Format Email Tidak Valid

Pengujian dilakukan dengan mengirimkan alamat email email-salah. API menolak data tersebut karena tidak memenuhi format email yang valid. Respons yang dihasilkan adalah status HTTP 400 Bad Request dengan pesan Format email tidak valid.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> $body = @{
>>     nim = "23005"
>>     nama = "Test"
>>     email = "email-salah"
>> } | ConvertTo-Json
>>
>> try {
>>     Invoke-RestMethod -Uri $uri -Method POST -ContentType "app
lication/json" -Body $body
>> } catch {
>>     $_.ErrorDetails.Message
>> }

{
    "success": false,
    "message": "Format email tidak valid",
    "data": null
}
```

3.6.4 Pengujian Format JSON Tidak Valid

Pengujian dilakukan dengan mengirimkan body request yang berisi JSON tidak lengkap. API berhasil mendeteksi kesalahan format tersebut dan mengembalikan status HTTP 400 Bad Request dengan pesan Format JSON tidak valid.

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 16 Restful API endpoint request response> $body = '{"nim": "2300
5", "nama": "Test",'
>>
>> try {
>>     Invoke-RestMethod -Uri $uri -Method POST -ContentType "app
lication/json" -Body $body
>> } catch {
>>     $_.ErrorDetails.Message
>> }

{
    "success": false,
    "message": "Format JSON tidak valid",
    "data": null
}
```

3.7 Rekapitulasi Hasil Pengujian

Berdasarkan pengujian yang dilakukan, diperoleh hasil sebagai berikut:

No.	Skenario Pengujian	Hasil
1	GET seluruh mahasiswa	Berhasil
2	GET mahasiswa berdasarkan ID	Berhasil
3	POST menambahkan mahasiswa	Berhasil
4	PUT memperbarui mahasiswa	Berhasil
5	DELETE menghapus mahasiswa	Berhasil
6	Mengakses ID yang tidak ditemukan	Berhasil ditangani
7	Mengirimkan NIM kosong	Berhasil ditangani
8	Mengirimkan email tidak valid	Berhasil ditangani
9	Mengirimkan JSON tidak valid	Berhasil ditangani

Hasil pengujian menunjukkan bahwa endpoint utama dapat menjalankan operasi pengelolaan data mahasiswa. API juga dapat mendeteksi input yang tidak valid dan mengembalikan respons kesalahan sesuai dengan skenario yang diuji.

BAB 4. PENUTUP

4.1 Kesimpulan

Berdasarkan pelaksanaan praktikum Acara 16, dapat disimpulkan bahwa RESTful API dapat diimplementasikan menggunakan PHP dan MySQL untuk melakukan pengelolaan data mahasiswa melalui metode HTTP GET, POST, PUT, dan DELETE.

Penggunaan format JSON memungkinkan data dikirim dan diterima secara terstruktur. Penggunaan PDO dan prepared statement membantu pelaksanaan query database secara terparameterisasi, sedangkan validasi input memastikan data yang tidak sesuai tidak diproses sebagai data yang valid.

Pengujian menunjukkan bahwa seluruh endpoint utama berhasil dijalankan dan API mampu menangani beberapa kondisi kesalahan, seperti ID yang tidak ditemukan, NIM kosong, format email tidak valid, dan format JSON yang salah. Dengan demikian, tujuan praktikum untuk memahami implementasi RESTful API, endpoint request/response, dan penanganan kesalahan telah tercapai.

4.2 Saran

Pengembangan selanjutnya dapat dilakukan dengan menambahkan autentikasi dan otorisasi pengguna, dokumentasi API yang lebih lengkap, pengujian otomatis, serta validasi tambahan terhadap setiap input. Selain itu, API dapat dikembangkan agar mendukung pagination dan mekanisme pencatatan kesalahan untuk mempermudah pemeliharaan aplikasi.